

When Is a Self-Improving Agent’s Diagnosis Worth Verifying? Evidence Allocation When Every Check Costs an Episode

Anonymous ACL submission

Abstract

Self-improving LLM agents let an analyst model read failed trajectories, blame a component, and patch it. Because attributions are unreliable, recent systems verify a patch before applying it: they replay the failure with the patch, ask an LLM judge, or re-run tasks that used to pass. Methods for this step usually assume that some evidence is cheap. We measured it on three tool-use settings, a fault-injected customer-service environment and the retail and airline domains of τ^2 -bench, using over 14,000 agent runs and judge checks. Every kind of evidence costs about one episode: a single-step judge check costs 23–72% of a full replay, and its sensitivity is only 0.37–0.57. We therefore recast verification as allocating a budget of episodes among targeted replays, null replays that measure spurious recovery, and regression runs, so as to maximise the net effect of the applied patches on success. A value-of-information argument shows that verification can pay only when a wrong patch does real harm: its value is bounded by $\min(\pi G, (1-\pi)H)$, where π is the chance that the patch is right, G its gain if right, and H its harm if wrong. On τ^2 -bench, wrong patches almost never hurt: only 3 of 121 patches significantly raise failures on previously solved tasks. Consistent with the bound, applying every attributed patch without evidence is as good as any verification policy on held-out tasks (success 0.76→0.88, against 0.83–0.88 for verified patch sets). When wrong changes carry a fixed cost, verification becomes useful, but no allocation policy dominates: the best one changes with domain and budget. A net-effect allocation that prices every experiment in episodes improves steadily with budget on both domains, whereas a multi-fidelity design method built on cheap checks degrades on airline as its budget grows. We release the environments, fault suites and intervention banks.

1 Introduction

A growing family of systems improves LLM agents from their own failures: the agent runs, an analyst model reads the failed trajectories, and its diagnosis drives a change to a prompt, a tool description or the orchestration (Shinn et al., 2023; Zhao et al., 2024; Hu et al., 2025; Zhang et al., 2025a; Agrawal et al., 2026; Chen et al., 2026). The diagnosis step, *failure attribution*, is unreliable (Zhang et al., 2025b; Liu et al., 2026): models label failures by their visible symptom rather than their cause, so a loop that trusts them will often change the wrong component. The common remedy is to verify before acting: patch the blamed component and replay the failure (Ma et al., 2025; Lin et al., 2026; Shah, 2026), ask a judge whether the failing step is fixed, or re-run tasks that used to pass and keep the patch only if nothing breaks (Chen et al., 2026).

Verification costs agent episodes, so the natural research question is how to spend them. A tempting answer, and the one we started from, treats LLM judgements as cheap evidence and full replays as expensive, and allocates a budget across both by Bayesian experimental design. In a simulator built on these assumptions that approach needs half the budget of the strongest baseline (Appendix D). On real agents the premise fails. Across three tool-use settings, a judge check costs 23–72% of a full replay, because the judge must read the trajectory, and it detects only 37–57% of real repairs. No evidence is cheap.

We therefore pose the problem with every piece of evidence priced in episodes, and ask a prior question: *when is a diagnosis worth verifying at all?* Our contributions are:

- A formulation of failure-driven self-improvement as *episode-priced evidence allocation*. A patch’s net effect on success decomposes into repair on its target category, spurious recovery that mimics repair, and

regressions on other tasks; targeted replays, null replays and regression runs each estimate one term (§3).

- A decision-theoretic bound: the value of verifying a patch is at most $\min(\pi G, (1 - \pi)H)$, so verification pays only when a wrong patch is both likely and harmful. Together with the sample sizes needed to estimate each term, this yields a test practitioners can run before spending a budget on verification (§4).
- Measurements on ShopDesk and offline intervention banks on τ^2 -bench retail and airline, with paired replays and judge checks for every patch, null replays, and 20 regression runs for every patch any method accepts. Evaluating eleven policies against them shows that on these agents wrong patches are nearly harmless, and that applying attributed patches without evidence matches every verification policy on held-out tasks, as the bound predicts (§7).
- A net-effect allocation method that models the three terms with per-category spurious recovery and chooses experiments by knowledge gradient per unit cost. When wrong changes are costly, its gain grows steadily with budget on both domains, although no policy, ours included, is best everywhere (§5, §7).

2 Related Work

Failure attribution for LLM agents. Zhang et al. (2025b) introduced the Who&When benchmark and compared all-at-once, step-by-step and binary-search prompting for locating the responsible agent and step; Liu et al. (2026) extended it to longer and multimodal traces and found that attribution remains far from solved. AgenTracer uses counterfactual replay and fault injection to build training data for a small attribution model (Zhang et al., 2026); AgentDebug pairs an error taxonomy with targeted corrective feedback (Zhu et al., 2025); ErrorProbe traces dependencies backwards and keeps a verified memory of error patterns (Li et al., 2026). Taxonomies such as MAST (Cemri et al., 2025) give attribution a fixed vocabulary, and AdaMAST induces a system-specific taxonomy from traces whose axes map to intervention points (Cemri et al., 2026). AdaMAST validates its taxonomy by agreement between LLM annotators; we instead treat the mapping from categories to components as a causal claim to be checked by intervention.

Verification by intervention. DoVer generates hypotheses from logs and validates each by editing a message or plan and re-running, with several runs per intervention (Ma et al., 2025). REFLECT tests diagnosis-specific patches by targeted replay and feeds verified outcome changes back into attribution (Lin et al., 2026). CausalFlow scores steps by their causal responsibility under counterfactual interventions (Bonagiri et al., 2026), and Causal Agent Replay formalises step interventions in a structural causal model with a point-of-commitment rule (Shah, 2026). AgentDebugX provides a toolkit with policy-gated reruns and a store of failure–diagnosis–repair bundles (Zhu et al., 2026). All of these operate per trajectory or per hypothesis and do not decide how to allocate a shared budget across hypotheses, patches and levels of fidelity, which is our focus. Our method can use any of them as the replay primitive.

Self-improving agents. Reflexion, Self-Refine and ExpeL turn feedback into verbal lessons (Shinn et al., 2023; Madaan et al., 2023; Zhao et al., 2024); prompt optimisers such as ProTeGi, OPRO, TextGrad and DSPy use failures or textual feedback to rewrite prompts (Pryzant et al., 2023; Yang et al., 2024; Yuksekogonul et al., 2025; Khattab et al., 2024); ADAS, the Darwin Gödel Machine and GEPA search over agent designs or prompts guided by evaluation and failure logs (Hu et al., 2025; Zhang et al., 2025a; Agrawal et al., 2026). Harness-Fix attributes failures to harness artefacts, repairs them, and promotes a repaired harness only if it solves more validation tasks while breaking at most two (Chen et al., 2026); its validation is regression-aware, but it uses whole-task runs on fixed splits rather than allocating evidence per patch. Bayesian-Agent keeps Beta posteriors over the reliability of skills and applies threshold rules to patch, split or retire them (Wu et al., 2026); its posterior is over skills rather than causal edges, and actions are chosen by fixed thresholds rather than by the value of information. CARVE is complementary: it decides which proposed fixes to trust before they are applied.

Experimental design. Our selection rule is the KG(*) variant of the knowledge gradient (Frazier et al., 2008; Frazier and Powell, 2010), within Bayesian experimental design (Chaloner and Verdinelli, 1995). Related problems include multi-fidelity bandits and optimisation (Kandasamy et al., 2016; Poloczek et al., 2017), thresholding and best-

arm identification (Locatelli et al., 2016; Audibert et al., 2010; Russo, 2016), sequential testing (Wald, 1945), active causal structure learning (Tong and Koller, 2001; Hauser and Bühlmann, 2014) and causal bandits (Lattimore et al., 2016). In software engineering, delta debugging isolates failure-inducing changes by experiment (Zeller and Hildebrandt, 2002), and program-repair work warns that a patch passing a failing test may not fix the fault (Smith et al., 2015), which our separation of edges from patch quality reflects. We use the knowledge gradient as an allocation rule, but our main question is a prior one from decision analysis: when the information is worth its cost at all.

3 Problem: Episode-Priced Evidence

Setting. An agent is assembled from *components* j (prompt sections, tool descriptions, configuration). On a task distribution it fails with rate F ; an LLM groups its failures into categories i with shares f_i . For each *cell* (i, j) that the analyst blames, it proposes K candidate patches k that rewrite component j . Let $e_{ij} \in \{0, 1\}$ indicate whether changing j can repair category i at all, and q_k the fraction of category- i failures that patch k repairs if so. Separating e from q matters because a patch that fails to help may be aimed at the wrong component or merely be a bad patch.

Net effect. Applying patch k of cell (i, j) changes the success rate by

$$\Delta_k = F f_i e_{ij} q_k - (1 - F) \rho_k - c, \quad (1)$$

where ρ_k is the increase in failure rate on tasks that used to succeed (regressions, which may hit any category) and $c \geq 0$ is a fixed cost of making a change (review, maintenance, risk), which we vary. The goal is to choose a set of patches that maximises $\sum_k \Delta_k$ on held-out tasks.¹

Evidence. Three kinds of experiment, each costing a fraction of an episode, inform Eq. 1:

- A *targeted replay* resumes a category- i failure from the attributed step with the patch applied; it succeeds with probability $b_i + (1 - b_i)e_{ij}q_k$.
- A *null replay* does the same without a patch and succeeds with probability b_i , the category’s *spurious recovery* rate. Without it, replay success cannot be interpreted.

¹Effects of patches to different components are treated as additive; we check this assumption on held-out tasks in §7.

- A *regression run* executes a previously successful task from scratch with the patch and fails with probability $r_0 + \rho_k$, where r_0 is the natural re-run failure rate.

A fourth, a *single-step check* that regenerates only the failing step and asks an LLM judge whether the error is gone, is often proposed as the cheap option. We measure its cost and accuracy in §7. Unlike classic multi-fidelity settings, all four have costs within a factor of four of each other.

4 When Can Verification Pay?

Consider one candidate patch. Before any evidence, let $\pi = P(e_{ij}=1)$ be the probability that it targets a real cause (for example, the analyst’s audited accuracy), $G > 0$ its expected net effect if it does, and $H \geq 0$ its expected harm if it does not, so $H = (1 - F)\bar{\rho}_{\text{wrong}} + c$.

Proposition 1 (Value of verification). *With no evidence, the best action is to apply the patch iff $\pi G \geq (1 - \pi)H$. The expected value of learning e_{ij} exactly is*

$$\text{VPI} = \min(\pi G, (1 - \pi)H), \quad (2)$$

and any verification policy, which observes noisy outcomes instead of e_{ij} , gains at most this much.

The proof is two lines (Appendix A). The bound has a direct reading. When wrong patches are harmless ($H \approx 0$), the default is to apply and verification has nothing to prevent; when the analyst is almost always right or almost always wrong, it has nothing to change. Verification is worth considering only when a patch is uncertain *and* a wrong one does damage. The second ingredient is what the evidence costs.

Proposition 2 (Cost of evidence). *Under Wald’s approximations, a sequential test that decides e_{ij} from targeted replays with error δ needs on average about $\log \frac{1-\delta}{\delta} / \text{KL}(b + (1-b)q \parallel b)$ replays when the edge is real. Detecting a regression of size ρ against a base rate r_0 with a one-sided level- α test and power $1 - \beta$ needs about $(z_\alpha + z_\beta)^2 r_0 (1 - r_0) / \rho^2$ regression runs.*

Two consequences follow. First, targeted replays are uninformative about H , because they only see the target category. A patch can pass every replay and still break other tasks, so verification that consists only of replays cannot capture the value in Proposition 1 when H comes from regressions.

Second, small regressions are expensive to see. With $r_0 = 0.1$, a regression of $\rho = 0.25$ is detected with 90% power by 20 runs, whereas $\rho = 0.03$ needs about 850. Verification is therefore most useful against rare, large harms, and it is the rate of such harms that decides whether it pays.

5 Allocation Policies

When verification is warranted, the budget must still be allocated across cells, patches and experiment types. We compare eleven policies.

Net-effect allocation (ours). We place a spike-and-slab model on each cell. The edge prior $P(e_{ij}=1) = \pi_{ij}$ comes from the analyst’s attribution counts, shrunk towards a base rate by its audited accuracy (Eq. 5 in Appendix D). Each q_k has a grid posterior from a Beta(2, 2) prior, each b_i a Beta posterior updated by null replays, and each ρ_k a Beta posterior updated by regression runs. Its estimate of Eq. 1 is $P(e_{ij}=1 \mid \mathcal{D}) F f_i \mathbb{E}[q_k \mid \mathcal{D}] - (1-F) \max(0, \mathbb{E}[r_k \mid \mathcal{D}] - r_0) - c$, where r_k is the patch’s failure rate in regression runs. The policy repeatedly runs the experiment (targeted replay of a patch, null replay of a category, or regression run of a patch) that maximises the n -step knowledge gradient of the decision value divided by its measured cost, with $n \in \{1, 2, 4, 8\}$ (Frazier et al., 2008), and finally applies every patch whose posterior net effect is positive. Unlike CARVE, it has no single-step checks, a separate b_i per category, and a regression term.

Baselines. *Apply attributed* uses no evidence: for each category it applies the first patch of the most-blamed component and of any component with at least 40% of the blame. *Replay-each* tests hypotheses in order of blame with three replays and accepts on two successes, as in do-then-verify methods (Ma et al., 2025). *HarnessFix-style, per patch* adds two regression runs that must break nothing; these counts are ours. *HarnessFix-style, bundle gate* is closer to the original, which validates a whole bundle of repairs on held-out tasks and promotes it if it solves at least one more task and breaks at most two (Chen et al., 2026). We apply the *Apply attributed* bundle, draw validation episodes from the bank, and allow three attempts. *Uncertainty sampling* replays the cell with the largest frequency-weighted posterior uncertainty, with or without three cheap checks per patch first. CARVE is our multi-fidelity design method (Appendix D); we also run it with

only full replays, with the judge’s measured accuracy, and with a quality-dependent false-positive rate.

6 Environments and Intervention Banks

Environments. *ShopDesk* is a customer-service environment we built, with a seeded database, six deterministic tools, exactly checkable answers and 11 named agent components. τ^2 -*bench retail and airline* (Yao et al., 2024; Barres et al., 2025) are dual-control customer-service domains with an LLM user simulator and database-state rewards. We split each domain policy into sections and add every tool description and the step limit as components, giving 24 components for retail and 21 for airline. The agent is DeepSeek-V4-Flash. Attribution, taxonomy induction (Wan et al., 2024), patch generation and judging use DeepSeek-V4-Pro.

Faults that look like defects. Each training episode runs clean or with one injected fault, written as a plausible authoring mistake rather than an obvious artefact. In retail: a policy section allowing gift-card refunds, a cancellation rule without the reason check, a product-tool description that confuses item and product ids, and a tight step limit. In airline: a wrong bag allowance, a 7-day instead of 24-hour cancellation window, a missing basic-economy restriction, proactive compensation, and a tight step limit. Clean episodes contribute natural failures: 22 of 137 on retail and 28 of 112 on airline.

Banks. For each τ^2 domain we take the 36 most-blamed (category, component) cells and generate three patches per cell. For every patch we record 20 paired targeted replays and single-step checks. We also record 20 null replays per category and 4 regression runs per patch, extended to 20 for every patch that any policy accepts (121 patches). Policies run offline by sampling outcomes from the bank, with costs in measured tokens and full replay = 1, over 200 seeds. *Ground truth needs no injected label*: an edge is real if its best patch’s replay success exceeds the category’s null rate by at least 0.2 (one-sided Fisher $p < 0.05$), and a patch counts as regressing if its failure rate on previously solved tasks exceeds r_0 (one-sided binomial $p < 0.05$). This yields 4 real edges on retail and 7 on airline. We report net gain (Eq. 1) as a fraction of the oracle that applies exactly the real edges’ best patches. We also apply the accepted patch sets to

Budget	τ^2 retail (development)						τ^2 airline (held out)					
	measured harm ($c=0$)			+ change cost c			measured harm ($c=0$)			+ change cost c		
	10	40	80	10	40	80	10	40	80	10	40	80
Apply attributed (no evidence)	0.84	0.84	0.84	0.16	0.16	0.16	0.60	0.60	0.60	0.15	0.15	0.15
Replay-each	0.78	0.74	0.69	0.14	0.33	0.34	0.43	0.03	0.05	0.07	-0.17	-0.13
HarnessFix-style, per patch	0.52	0.48	0.49	0.52	0.39	0.27	0.18	0.40	0.43	0.18	0.37	0.31
HarnessFix-style, bundle gate	0.07	0.16	0.20	-0.24	-0.36	-0.36	0.34	0.53	0.52	0.07	0.12	0.11
Uncertainty sampling	0.37	0.46	0.25	-0.04	0.19	-0.06	0.55	0.48	0.49	0.27	0.23	0.23
Uncertainty + cheap checks	0.06	0.43	0.44	-0.33	0.21	0.21	0.55	0.50	0.53	0.27	0.29	0.35
CARVE, full replay only	-0.09	0.10	0.05	-0.58	-0.34	-0.48	0.15	0.30	0.44	-0.23	-0.07	0.09
CARVE	0.27	0.27	0.51	-0.06	0.15	0.46	0.55	0.34	0.25	0.30	0.23	0.19
CARVE, measured check	-0.09	0.10	0.05	-0.58	-0.34	-0.48	0.19	0.39	0.52	-0.15	0.04	0.18
CARVE, robust check	0.28	-0.15	-0.09	-0.21	-0.60	-0.59	0.15	0.30	0.45	-0.19	-0.06	0.11
Net-effect allocation (ours)	0.17	0.49	0.64	0.02	0.38	0.50	0.22	0.37	0.44	0.10	0.28	0.33

Table 1: Net gain as a fraction of the oracle on the τ^2 intervention banks, mean over 200 resampling seeds; bold is the best per column. “Measured harm” charges each accepted patch its significant measured regression; “+ change cost” also charges $c = 0.02$ of the failure mass per wrong change. Standard errors are at most 0.08 (retail) and 0.03 (airline); they reflect resampling only, not the finite bank.

	ShopDesk	Retail	Airline	No evidence is cheap. A judge check costs 23–72% of a full replay, because the judge reads the trajectory, and it recognises only 37–57% of real repairs. Its information per unit cost is below that of a replay in every environment: the break-even cost ratio is about 0.1, and the measured ratio is 0.23 or more. Its false-positive rate is no higher for bad patches than for good ones, so the failure mode we feared, a judge fooled by plausible bad patches, did not occur. The judge is weak, not misled.
Analyst accuracy $\hat{r}$	0.72	0.41	0.29	
Real edges among blamed cells	–	0.18	0.33	
Judge check: cost / replay	0.72	0.28	0.23	
sensitivity	0.57	0.37	0.41	
false-positive rate	0.21	0.11	0.13	
break-even cost ratio	0.08–0.13	0.09	0.10	
Spurious recovery b (mean)	–	0.23	0.20	
range over categories	–	0–0.50	0–0.50	
Repair q of real edges (median)	–	0.50	0.41	
Replays to decide an edge (Prop. 2)	–	5	7	
Natural re-run failure r_0	–	0.11	0.14	
Patches regressing ($p < 0.05$)	–	2/61	1/60	
Excess failure, wrong patches	–	+0.03	–0.03	

Table 2: Measured quantities. “Real edges among blamed cells” is the precision of applying every attributed patch. The break-even cost ratio is the cost below which judge checks are more informative per unit cost than replays (Appendix D). Excess failure is relative to r_0 ; both have a standard error of about 0.01, and r_0 itself is uncertain by 0.03–0.04.

Replays need controls. Without any patch, 20–23% of replays from the failing step succeed anyway, and the rate ranges from 0 to 0.5 across categories. A replay success rate of 0.5 is strong evidence in one category and none in another. Deciding one edge takes 5–7 replays even with the right b_i .

40 unseen retail tasks.

Protocol. Retail was the development domain for every method, including the variants in Appendix B. Airline was evaluated once for the original policies and once, later, for the net-effect method, which was frozen on retail beforehand.

7 Results

7.1 What evidence costs and what it shows

Table 2 reports the quantities that Propositions 1 and 2 depend on.

Wrong patches are nearly harmless. With 20 regression runs per patch, only 3 of 121 patches raise failures on previously solved tasks significantly, about what chance alone would produce at $p < 0.05$. On average, wrong patches change that failure rate by +0.03 on retail and –0.03 on airline, within the uncertainty of r_0 . The exception we observed is on ShopDesk: a formatting patch (“reply FINAL: N/A rather than guessing”) made 26% of held-out tasks answer N/A; removing it raises held-out success from 0.73 to 0.89. It passed its target-category replays, which by construction cannot see such harm (§4).

Patches chosen at budget 10	Success	#patches
Unpatched agent	0.76 ± 0.04	–
Apply attributed (no evidence)	0.88 ± 0.00	7.0
Replay-each	0.83 ± 0.01	6.0
HarnessFix-style, per patch	0.83 ± 0.01	0.8
HarnessFix-style, bundle gate	0.85 ± 0.02	5.6
Uncertainty sampling	0.86 ± 0.01	5.8
Uncertainty + cheap checks	0.85 ± 0.01	5.8
CARVE, full replay only	0.88 ± 0.01	6.0
CARVE	0.84 ± 0.01	5.2
CARVE, measured check	0.85 ± 0.01	6.0
CARVE, robust check	0.86 ± 0.01	6.0
Net-effect allocation (ours)	0.83 ± 0.01	3.4

Table 3: Held-out success on retail: 40 unseen tasks $\times$ 3 episodes (clean and fault-injected, as in training) with each policy’s accepted patches, mean $\pm$ s.e. over 5 seeds. Episode noise is about ± 0.03 per patch set.

7.2 Does verification pay?

Plugging the measurements into Proposition 1: with $H \approx 0$, the value of verifying a blamed patch is close to zero, so the best policy should be to apply what the analyst blames. Two independent measurements agree.

Held-out tasks. Table 3 applies each policy’s patch set to 40 unseen retail tasks. Every set lifts success from 0.76 to 0.83–0.88, and applying the attributed patches without any evidence is among the best (0.88 with seven patches, most of them aimed at wrong edges). The verified sets are smaller and no better. Most of the gain comes from one patch, the step-limit fix, which almost every policy accepts.

Intervention banks. The left half of each block in Table 1 charges only measured regressions ($c = 0$). Applying the attributed patches recovers 84% (retail) and 60% (airline) of the oracle’s gain with no budget; no verification policy reaches it at any budget, because each rejects some real edges whose replays it cannot distinguish from spurious recovery within the budget. This is the regime of Proposition 1 with $H \approx 0$: rejecting a real edge costs gain, while accepting a wrong one costs nothing.

7.3 When wrong changes cost something

A deployment may still attach a cost to every change, for review, maintenance or risk that 20 regression runs cannot see. The right half of each block in Table 1 charges $c = 0.02$ of the failure mass per wrong change. Applying everything now drops to 16% and 15%, as its eight or nine wrong patches are charged, and verification pays. Which

policy is best now depends on domain and budget. On retail, the per-patch HarnessFix-style rule wins at budgets 10 and 40 (0.52, 0.39) because the most valuable edge is also the most blamed, so testing in order of blame finds it at once; the net-effect method is best at 80 (0.50). On airline, CARVE leads at budget 10 (0.30), HarnessFix-style at 40 (0.37), and uncertainty sampling with checks at 80 (0.35, against 0.33 for the net-effect method).

Two patterns are robust. The net-effect method improves with budget on both domains (retail $0.02 \rightarrow 0.50$, airline $0.10 \rightarrow 0.33$), whereas CARVE *declines* on airline ($0.30 \rightarrow 0.19$): with one shared spurious-recovery rate, further replays in high- b categories push it to reject real edges. Second, cheap checks earn their place only by accident. With its default, overconfident check model, CARVE beats its full-replay variant, but with the measured accuracy it runs almost no checks and stays close to its full-replay variant. The bundle gate that is closest to HarnessFix’s own protocol is the weakest verifier under a change cost (-0.36 to 0.12). It validates the analyst’s whole bundle, so one real fix lets several wrong ones through.

7.4 Why the simulator was optimistic

Our first design study used a simulator in which CARVE needed half the budget of the strongest baseline (Appendix D). Re-running it with the measured judge accuracy, spurious recovery, analyst accuracy and cost ratio still predicts that CARVE leads uncertainty sampling by 14 points on retail at budget 50. On the bank, the comparable variant (CARVE with the measured check, scored with the change cost and without regressions) trails it by 15 points at budget 40. The simulator shared the method’s structure: one b for all categories, value spread over many edges, and a fixed regression cost c that made every wrong change harmful. The last assumption alone decides whether verification pays (Proposition 1), and it was never measured.

8 Discussion

Measure H first. The bound in Proposition 1 makes the harm of a wrong patch the first quantity to measure, before choosing a verification method. Measuring it needs regression runs, not targeted replays or judges, and with $r_0 \approx 0.1$ about 20 runs per patch reveal harms of 0.25 or more. On our agents such harms were rare, and verification policies mostly removed real fixes. Where wrong

changes are costly (production agents, safety-relevant behaviour, changes reviewed by people), verification pays, and allocation matters.

Recommendations. (i) Price every kind of evidence in measured tokens; judge checks were never cheap enough to help. (ii) Run null replays per category before interpreting replay success. (iii) Spend regression runs on screening for large cross-category harms. (iv) If changes are cheap and rarely harmful, apply attributed patches and let the next round of failures correct them.

9 Conclusion

We set out to make verification of LLM failure attributions cheaper and found that, on real agents, no verification evidence is cheap and wrong patches rarely hurt. Treating evidence as episode-priced and asking what verification can be worth at all shows that its value is capped by the harm it prevents. On our three tool-use settings that harm was close to zero, and acting on attributions without evidence matched every verification policy. Verification pays when wrong changes are costly, and then the allocation policy matters, though no single policy is best everywhere. We release the environments, fault suites and banks so that verification methods can be judged against measured interventions rather than simulators.

Limitations

We study one agent model and one analyst model on three customer-service settings. The banks have 36 cells per domain and only 4 and 7 real edges, so individual edges weigh heavily, and resampling standard errors understate the uncertainty of a finite bank. Ground truth is defined by tests on 20 replays and 20 regression runs, which cannot see small effects. Regressions smaller than about 0.1 are invisible at this sample size, yet could accumulate over many patches. Held-out success was measured on retail at budget 10 only, and the patch sets of the earlier policies were chosen before the regression runs were extended. Most failures come from injected faults written to resemble real defects; natural failures are 16–25%. The net-effect method was developed on retail after we had seen airline results for the other policies, although not its own. Finally, replay from a failing step requires deterministic tools.

Ethics Statement

Self-improving agents change their own behaviour without a person reviewing each change. Our finding that wrong patches were rarely harmful holds for the agents and tasks we studied and should not be read as licence to skip review elsewhere. The ShopDesk example shows a single patch silently breaking a quarter of tasks. Our environments use synthetic customer data only.

References

- Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations*.
- Jean-Yves Audibert, Sébastien Bubeck, and Rémi Munos. 2010. Best arm identification in multi-armed bandits. In *Proceedings of the 23rd Conference on Learning Theory*.
- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. τ^2 -bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*.
- Akash Bonagiri, Devang Borkar, Gerard Janno Anderson, Setareh Rafatirad, and Houman Homayoun. 2026. CausalFlow: Causal attribution and counterfactual repair for LLM agent failures. *arXiv preprint arXiv:2605.25338*.
- Mert Cemri, Andrei Cojocaru, Melissa Pan, Shu Liu, Shubham Agarwal, Alexander Krentzel, Jay Tang, Kannan Ramchandran, Joseph E. Gonzalez, Matei Zaharia, Alexandros G. Dimakis, and Ion Stoica. 2026. Fantastic adaptive taxonomies and how to use them. *arXiv preprint arXiv:2607.16387*.
- Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why do multi-agent LLM systems fail? *arXiv preprint arXiv:2503.13657*.
- Kathryn Chaloner and Isabella Verdinelli. 1995. Bayesian experimental design: A review. *Statistical Science*, 10(3):273–304.
- Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. 2026. From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws. *arXiv preprint arXiv:2606.06324*.

596	Peter I. Frazier and Warren B. Powell. 2010. Paradoxes	649
597	in learning and the marginal value of information.	650
598	<i>Decision Analysis</i> , 7(4):378–403.	651
599	Peter I. Frazier, Warren B. Powell, and Savas Dayanik.	652
600	2008. A knowledge-gradient policy for sequential	653
601	information collection. <i>SIAM Journal on Control</i>	654
602	<i>and Optimization</i> , 47(5):2410–2439.	655
603	Alain Hauser and Peter Bühlmann. 2014. Two opti-	656
604	mal strategies for active learning of causal models	
605	from interventional data. <i>International Journal of</i>	
606	<i>Approximate Reasoning</i> , 55(4):926–939.	
607	Shengran Hu, Cong Lu, and Jeff Clune. 2025. Auto-	
608	maticated design of agentic systems. In <i>International</i>	
609	<i>Conference on Learning Representations</i> .	
610	Kirthevasan Kandasamy, Gautam Dasarathy, Jeff	
611	Schneider, and Barnabás Póczos. 2016. The multi-	
612	fidelity multi-armed bandit. In <i>Advances in Neural</i>	
613	<i>Information Processing Systems</i> .	
614	Omar Khattab, Arnav Singhvi, Paridhi Maheshwari,	
615	Zhiyuan Zhang, Keshav Santhanam, Sri Vard-	
616	hamanan, Saiful Haq, Ashutosh Sharma, Thomas T.	
617	Joshi, Hanna Moazam, Heather Miller, Matei Zaharia,	
618	and Christopher Potts. 2024. DSPy: Compiling	
619	declarative language model calls into self-improving	
620	pipelines. In <i>International Conference on Learning</i>	
621	<i>Representations</i> .	
622	Finnian Lattimore, Tor Lattimore, and Mark D. Reid.	
623	2016. Causal bandits: Learning good interventions	
624	via causal inference. In <i>Advances in Neural Informa-</i>	
625	<i>tion Processing Systems</i> .	
626	Jiazheng Li, Emine Yilmaz, Bei Chen, and Dieu-	
627	Thu Le. 2026. Towards self-improving error di-	
628	agnosis in multi-agent systems. <i>arXiv preprint</i>	
629	<i>arXiv:2604.17658</i> .	
630	Xiaofeng Lin, Yingxu Wang, Tung Sum Thomas Kwok,	
631	Daniel Guo, Sahil Arun Nale, Charles Fleming,	
632	and Guang Cheng. 2026. REFLECT: Intervention-	
633	supported error attribution for silent failures in LLM	
634	agent traces. <i>arXiv preprint arXiv:2606.09071</i> .	
635	Jiale Liu, Huajun Xi, Shaokun Zhang, Yifan Zeng, Tian-	
636	wei Yue, Chi Wang, Jian Kang, Qingyun Wu, and	
637	Huazheng Wang. 2026. Who&when pro: Can LLMs	
638	really attribute failures in AI agents? <i>arXiv preprint</i>	
639	<i>arXiv:2607.09996</i> .	
640	Andrea Locatelli, Maurilio Gutzeit, and Alexandra Car-	
641	pentier. 2016. An optimal algorithm for the thresh-	
642	olding bandit problem. In <i>Proceedings of the 33rd</i>	
643	<i>International Conference on Machine Learning</i> .	
644	Ming Ma, Jue Zhang, Fangkai Yang, Yu Kang, Qing-	
645	wei Lin, Saravan Rajmohan, and Dongmei Zhang.	
646	2025. DoVer: Intervention-driven auto debug-	
647	ging for LLM multi-agent systems. <i>arXiv preprint</i>	
648	<i>arXiv:2512.06749</i> .	
	Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler	649
	Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon,	650
	Nouha Dziri, Shrimai Prabhumoye, Yiming Yang,	651
	Shashank Gupta, Bodhisattwa Prasad Majumder,	652
	Katherine Hermann, Sean Welleck, Amir Yazdan-	653
	bakhsh, and Peter Clark. 2023. Self-refine: Iterative	654
	refinement with self-feedback. In <i>Advances in Neu-</i>	655
	<i>ral Information Processing Systems</i> .	656
	Matthias Poloczek, Jialei Wang, and Peter I. Frazier.	657
	2017. Multi-information source optimization. In	658
	<i>Advances in Neural Information Processing Systems</i> .	659
	Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chen-	660
	guang Zhu, and Michael Zeng. 2023. Automatic	661
	prompt optimization with “gradient descent” and	662
	beam search. In <i>Proceedings of the 2023 Confer-</i>	663
	<i>ence on Empirical Methods in Natural Language</i>	664
	<i>Processing</i> .	665
	Daniel Russo. 2016. Simple Bayesian algorithms for	666
	best arm identification. In <i>Proceedings of the 29th</i>	667
	<i>Conference on Learning Theory</i> .	668
	Jaineet Shah. 2026. Causal agent replay: Counterfactual	669
	attribution for LLM-agent failures. <i>arXiv preprint</i>	670
	<i>arXiv:2606.08275</i> .	671
	Noah Shinn, Federico Cassano, Ashwin Gopinath,	672
	Karthik Narasimhan, and Shunyu Yao. 2023. Re-	673
	flexion: Language agents with verbal reinforcement	674
	learning. In <i>Advances in Neural Information Pro-</i>	675
	<i>cessing Systems</i> .	676
	Edward K. Smith, Earl T. Barr, Claire Le Goues, and	677
	Yuriy Brun. 2015. Is the cure worse than the disease?	678
	overfitting in automated program repair. In <i>Proceed-</i>	679
	<i>ings of the 10th Joint Meeting on Foundations of</i>	680
	<i>Software Engineering</i> .	681
	Simon Tong and Daphne Koller. 2001. Active learning	682
	for structure in Bayesian networks. In <i>Proceedings of</i>	683
	<i>the 17th International Joint Conference on Artificial</i>	684
	<i>Intelligence</i> .	685
	Rakshith Vasudev, Melisa Russak, Dan Bikel, and	686
	Waseem Alshikh. 2026. Accurate failure prediction	687
	in agents does not imply effective failure prevention.	688
	<i>arXiv preprint arXiv:2602.03338</i> .	689
	Abraham Wald. 1945. Sequential tests of statistical	690
	hypotheses. <i>The Annals of Mathematical Statistics</i> ,	691
	16(2):117–186.	692
	Mengting Wan, Tara Safavi, Sujay Kumar Jauhar, Yujin	693
	Kim, Scott Counts, Jennifer Neville, Siddharth Suri,	694
	Chirag Shah, Ryen W. White, Longqi Yang, Reid	695
	Andersen, Georg Buscher, Dhruv Joshi, and Nagu	696
	Rangan. 2024. TnT-LLM: Text mining at scale with	697
	large language models. In <i>Proceedings of the 30th</i>	698
	<i>ACM SIGKDD Conference on Knowledge Discovery</i>	699
	<i>and Data Mining</i> .	700
	Xiaojun Wu, Cehao Yang, Honghao Liu, Xueyuan Lin,	701
	Wenjie Zhang, Zhichao Shi, Xuhui Jiang, Chengjin	702
	Xu, Jia Li, and Jian Guo. 2026. Bayesian-Agent:	703

Posterior-guided skill evolution across LLM agent harnesses. *arXiv preprint arXiv:2606.08348*.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large language models as optimizers. In *International Conference on Learning Representations*.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*.

Mert Yuksekogonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. Optimizing generative AI by back-propagating language model feedback. *Nature*, 639.

Andreas Zeller and Ralf Hildebrandt. 2002. Simplifying and isolating failure-inducing input. *IEEE Transactions on Software Engineering*, 28(2):183–200.

Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. 2026. AgentTracer: Who is inducing failure in the LLM agentic systems? In *International Conference on Learning Representations*. ArXiv:2509.03312.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. 2025a. Darwin Gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*.

Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. 2025b. Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. In *Proceedings of the 42nd International Conference on Machine Learning*. ArXiv:2505.00212.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, Jiaxun Zhang, Pengrui Han, Qipeng Xie, Fuyang Cui, Weijia Zhang, Xiaoteng Ma, Xiaodong Yu, Gowtham Ramesh, Jialian Wu, Zicheng Liu, Pan Lu, James Zou, and Jiaxuan You. 2025. Where LLM agents fail and how they can learn from failures. *arXiv preprint arXiv:2509.25370*.

Kunlun Zhu, Xuyan Ye, Zhiguang Han, Yuchen Zhao, Bingxuan Li, Weijia Zhang, Muxin Tian, Xiangru Tang, Pan Lu, James Zou, Jiaxuan You, and Heng Ji. 2026. AgentDebugX: An open-source toolkit for failure observability, attribution, and recovery in LLM agents. *arXiv preprint arXiv:2607.18754*.

A Proofs

Proposition 1. Without evidence the expected value of applying is $\pi G - (1 - \pi)H$ and of not applying is 0, so the default value is $\max(\pi G - (1 - \pi)H, 0)$. Knowing e_{ij} , one applies iff $e_{ij} = 1$, for expected value πG . The difference is $\pi G - \max(\pi G - (1 - \pi)H, 0) = \min(\pi G, (1 - \pi)H)$. A verification policy observes outcomes that are a garbling of e_{ij} ; by the data-processing property of decision value (Blackwell), its value is at most that of observing e_{ij} , before subtracting the cost of its experiments.

Proposition 2. The first part is Wald’s approximation for a sequential probability ratio test between $p_1 = b + (1 - b)q$ and $p_0 = b$: with thresholds $\pm \log \frac{1-\delta}{\delta}$, neglecting overshoot, the expected sample size under p_1 is the threshold divided by the per-observation drift $\text{KL}(p_1 \| p_0)$. The second is the normal-approximation sample size of a one-sided test of a binomial proportion, evaluated with the variance at r_0 .

B Development Variants

On the retail bank we tried, before any airline evaluation of them:

- CARVE with two null replays per category before trusting its replays (per-category b);
- CARVE with a regression gate of two runs per accepted patch;
- both combined;
- an evidence rule that requires a positive likelihood ratio from data;
- tempered and flat priors;
- a Bayesian regression gate.

With control replays and the gate combined, CARVE gained 0.68 and 0.60 of the oracle at budgets 10 and 20, above plain CARVE (0.42, 0.48), but fell below it at 40 and 80 (0.47 vs. 0.57 and 0.56). It never beat the per-patch HarnessFix-style rule (0.72, 0.73). Post hoc on airline it was worse than plain CARVE (0.19 vs. 0.31 at budget 10). The other variants were worse everywhere; the evidence rule and the Bayesian gate accepted almost nothing (gain ≤ 0.12). These numbers use the fixed change cost c without regressions and the bank before the regression runs were extended. The net-effect method of §5 was developed afterwards, on retail, from the diagnosis that CARVE lacked

per-category spurious recovery and evidence for regressions.

C ShopDesk Details

ShopDesk has six deterministic tools (orders, products, policies, refunds), answers checked exactly (refund amounts, counts, totals), and five injected faults: a tool description misstating price units, a list tool whose description omits pagination, a format instruction that rounds to whole dollars, a planning instruction that skips the policy lookup, and a tight step limit. Over three seeds, the analyst’s attribution accuracy on injected failures was 0.75, 0.64 and 0.77. ShopDesk was evaluated online, without a bank: each policy ran against the live agent with fault-based ground truth (an edge is real when at least 30% of a category’s failures come from that component’s fault). Table 4 reports edge F1 and held-out success.

D The CARVE Design Study

Our first approach treated judge checks as cheap and replays as expensive, and allocated a budget over both by Bayesian experimental design. We keep it here as the multi-fidelity method compared in §7, together with the simulator in which it was developed. The simulator shares the method’s structure. Its results show the method working under its own assumptions, and they are not evidence about real agents.

Calibrated simulation. With the measured judge accuracy, spurious recovery, analyst accuracy and cost ratio, the simulator predicts that CARVE runs no judge checks on ShopDesk and retail and only a few on airline, which matches the break-even analysis in Table 2. For τ^2 retail it predicts net gains at budget 50 of 0.51 for CARVE and 0.38 for uncertainty sampling; for airline, 0.33 and 0.31; for ShopDesk, 0.81 and 0.79.

D.1 Problem Formulation

An agent is built from *components* $j \in \{1, \dots, J\}$: sections of its system prompt, tool descriptions, tool implementations, orchestration settings. Its failures have been grouped into *categories* $i \in \{1, \dots, I\}$ by an LLM, with f_i the share of failures in category i . For every cell (i, j) an analyst LLM can propose K candidate patches a_{ijk} that rewrite component j to address category i .

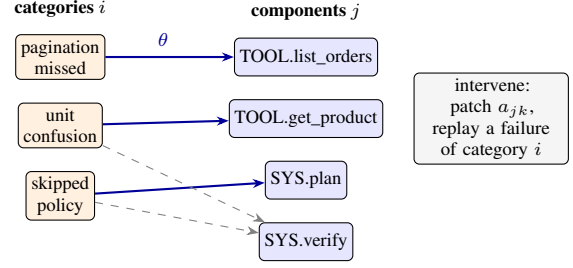


Figure 1: The category–component graph. Solid edges are true causal edges; dashed ones are plausible but wrong attributions (e.g. blaming verification for a unit error). Each intervention patches one component and replays one failure.

Interventions. An intervention (i, j, k, ϕ) samples a failed trajectory labelled i , applies patch a_{ijk} , and observes a binary outcome at fidelity ϕ . A *full replay* ($\phi = F$) resumes the episode from the attributed failing step and records whether the task now succeeds. A *single-step check* ($\phi = S$) regenerates only the failing step and asks a judge whether the diagnosed error is gone. They cost c_F and $c_S \ll c_F$.

Edges and patch quality. We write $e_{ij} \in \{0, 1\}$ for whether component j is a cause of category i , meaning some change to j repairs a non-trivial fraction of its failures, and $q_{ijk} \in [0, 1]$ for the repair probability of patch k when $e_{ij} = 1$. This separation matters because a negative replay outcome is ambiguous: either j is the wrong component or a_{ijk} is a bad patch. Testing several patches per cell resolves the ambiguity.

Objective. After spending at most budget B , the method accepts a set of cells $\mathcal{A}$ and, for each, a patch k_{ij}^* . Accepting a true edge repairs a fraction q_{ijk^*} of category i ’s failures; accepting a false edge costs a regression c_{fp} , reflecting that unnecessary changes perturb behaviour on tasks that used to succeed (Vasudev et al., 2026). We measure

$$G(\mathcal{A}) = \sum_i f_i \left(1 - \prod_{j \in \mathcal{A}_i^+} (1 - q_{ijk_{ij}^*}) \right) - c_{fp} |\{(i, j) \in \mathcal{A} : e_{ij} = 0\}|, \quad (3)$$

where $\mathcal{A}_i^+ = \{j : (i, j) \in \mathcal{A}, e_{ij} = 1\}$. This is the net failure mass repaired; we report it as a fraction of the oracle value obtained by accepting exactly the true edges with their best patches.

Method	Edge F1 at budget				Held-out success @60	Single checks
	10	20	30	60		
LLM attribution only	0.61 $\pm$ 0.04	0.61 $\pm$ 0.04	0.61 $\pm$ 0.04	0.61 $\pm$ 0.04	0.98 $\pm$ 0.00	0
Replay-each (do-then-verify)	0.62 $\pm$ 0.03	0.63 $\pm$ 0.03	0.67 $\pm$ 0.05	0.75 $\pm$ 0.03	0.99 $\pm$ 0.00	0
Uncertainty sampling	0.55 $\pm$ 0.04	0.57 $\pm$ 0.04	0.58 $\pm$ 0.05	0.66 $\pm$ 0.04	0.99 $\pm$ 0.00	0
Uncertainty + cheap checks	0.52 $\pm$ 0.07	0.56 $\pm$ 0.07	0.53 $\pm$ 0.09	0.53 $\pm$ 0.08	0.96 $\pm$ 0.03	57
CARVE, full replay only	0.61 $\pm$ 0.07	0.61 $\pm$ 0.06	0.59 $\pm$ 0.04	0.64 $\pm$ 0.03	0.91 $\pm$ 0.09	0
CARVE (default check model)	0.59 $\pm$ 0.06	0.57 $\pm$ 0.05	0.61 $\pm$ 0.05	0.68 $\pm$ 0.03	0.98 $\pm$ 0.01	4
CARVE, calibrated check	0.59 $\pm$ 0.06	0.58 $\pm$ 0.05	0.60 $\pm$ 0.05	0.66 $\pm$ 0.04	0.89 $\pm$ 0.08	0
CARVE, robust check model	0.58 $\pm$ 0.07	0.61 $\pm$ 0.06	0.61 $\pm$ 0.04	0.69 $\pm$ 0.01	0.99 $\pm$ 0.01	0
Unpatched agent	0.59 $\pm$ 0.04					

Table 4: ShopDesk (3 seeds, mean $\pm$ s.e.). Held-out success after applying the patches accepted at budget 60 to 120 held-out episodes.

D.2 CARVE

Observation model For a trajectory labelled i and patch a_{ijk} , the probability that a full replay succeeds is

$$p_{ijk} = \begin{cases} \lambda q_{ijk} + (1 - \lambda) b & e_{ij} = 1, \\ b & e_{ij} = 0, \end{cases} \quad (4)$$

where b is the spurious recovery rate (stochastic re-generation sometimes succeeds regardless of the patch) and λ is the accuracy of the category labels used to sample trajectories; a mislabelled trajectory is treated as unaffected by the patch. A single-step check returns $z = 1$ with probability $sp + \epsilon(1 - p)$, where s and ϵ are the sensitivity and false-positive rate of the check relative to the full outcome. All four nuisance parameters can be estimated from a small calibration set on which both fidelities and a human audit are run.

Attribution-informed prior Let n_{ij} be the number of failures in category i that the analyst attributed to component j , and $\hat{r}$ the analyst’s attribution accuracy estimated on a small audited sample. With d the expected number of causal components per category and $\rho = d/J$ the base edge density, we set

$$\pi_{ij} = (1 - \hat{r})\rho + \hat{r} \min(1, d n_{ij}/n_i), \quad (5)$$

so that an unreliable analyst ($\hat{r} \rightarrow 0$) yields the flat prior ρ and a reliable one concentrates mass on the components it names. Eq. 5 is a heuristic interpolation, not a calibrated model; with a larger audit, π could instead be fitted by logistic regression on audited attributions. Patch quality has prior $q_{ijk} \sim \text{Beta}(\alpha_q, \beta_q)$.

Posterior Cells are independent given the data. For each cell we keep the posterior log-odds of e_{ij}

and, for each patch, a posterior over q_{ijk} on a fixed grid of $G = 101$ points. With $\mathcal{D}_{ijk}$ the outcomes observed for patch k ,

$$\text{logit } P(e_{ij}=1 \mid \mathcal{D}) = \text{logit } \pi_{ij} + \sum_k \log(\bar{L}_{ijk}^1 / L_{ijk}^0), \quad (6)$$

where $\bar{L}_{ijk}^1 = \int P(\mathcal{D}_{ijk} \mid q, e_{ij}=1) dP(q)$ is the marginal likelihood of patch k ’s outcomes under a true edge and $L_{ijk}^0 = P(\mathcal{D}_{ijk} \mid e_{ij}=0)$. Every update multiplies one grid vector by a likelihood vector, so updates and look-aheads are vectorised dot products.

Selecting interventions The value of the current decision for cell (i, j) , with P_{ij} its edge posterior and $\hat{Q}_{ij} = \max_k \mathbb{E}[q_{ijk} \mid \mathcal{D}]$, is

$$V_{ij} = \max(P_{ij} f_i \hat{Q}_{ij} - (1 - P_{ij}) c_{\text{fp}}, 0), \quad (7)$$

the expected contribution of the cell to Eq. 3 under the Bayes decision (accept iff the first term is positive), treating cells as additive. Eq. 3 combines components of one category multiplicatively, so V_{ij} slightly overstates the value of a second cause for an already repaired category; we accept this approximation because it keeps cells independent. For an action $a = (i, j, k, \phi)$ repeated n times, let $V'_{ij}(s)$ be the value after observing s successes. The n -step knowledge gradient is

$$\text{KG}_n(a) = \sum_{s=0}^n P(s \mid \mathcal{D}, a) V'_{ij}(s) - V_{ij}, \quad (8)$$

and CARVE runs the action maximising

$$\text{score}(a) = \max_{n \in \{1, 2, 4, 8\}} \frac{\text{KG}_n(a)}{n c_\phi}. \quad (9)$$

Looking ahead several observations matters because a single binary outcome often cannot flip

a decision, in which case $KG_1 = 0$ even for a valuable cell; maximising over n is the standard remedy (Frazier et al., 2008). Because cells are independent, only the observed cell’s scores change after each outcome, so each step rescores one cell in $O(KG)$ time after an initial pass over all cells.

Decisions and stopping. At any point the method accepts the cells whose current value is positive and applies, for each, the patch with the highest posterior mean quality. It stops early when no action has a score above a small threshold, i.e. when no remaining experiment is expected to be worth its cost.

D.3 When Do Priors and Cheap Checks Help?

These results analyse an idealised per-cell sequential test, not CARVE’s full policy; they explain which quantities govern the method’s behaviour. We consider a single cell and a single patch with known success probabilities p_1 (true edge) and $p_0 = b$, under a rule that stops when the posterior log-odds leaves $[-A, A]$, $A = \log \frac{1-\delta}{\delta}$. Let $D_1 = \text{KL}(p_1||p_0)$ and $D_0 = \text{KL}(p_0||p_1)$ be the expected log-likelihood-ratio drift per observation under each hypothesis. Proofs, which follow Wald’s approximations (Wald, 1945), are in Appendix D.5.

Proposition 3 (Value of the LLM prior). *Let $|\ell| < A$ and neglect overshoot and stopping at the wrong boundary (whose probability is approximately $\delta e^{-\ell}$ for a true edge and δe^{ℓ} for a false one). A cell with initial log-odds ℓ then needs $N_1(\ell) = (A - \ell)/D_1$ observations in expectation if it is a true edge and $N_0(\ell) = (A + \ell)/D_0$ if not. Replacing a flat prior ℓ_ρ by LLM-derived log-odds ℓ_{ij} saves, in expectation over cells,*

$$\Delta = \frac{\rho \mathbb{E}[\delta\ell \mid e=1]}{D_1} - \frac{(1-\rho) \mathbb{E}[\delta\ell \mid e=0]}{D_0}, \quad (10)$$

with $\delta\ell = \ell_{ij} - \ell_\rho$, observations per cell, which is positive exactly when the prior raises true edges more, relative to their base rate and drift, than it raises false ones.

Under Eq. 5, $\ell_{ij} \rightarrow \ell_\rho$ as $\hat{r} \rightarrow 0$, so $\Delta \rightarrow 0$: a useless analyst costs nothing beyond the flat prior, whereas trusting its labels outright (LLM-only) makes permanent errors. A miscalibrated analyst can make Δ negative, but only by an amount linear in the log-odds shift, after which the evidence takes over. Appendix D.7 compares these predictions with simulated sequential tests.

Proposition 4 (Cheap checks). *Let D_e^S denote the drifts of single-step outcomes, $z \sim \text{Bern}(s p_e + \epsilon(1 - p_e))$. Under the same approximations, a test using only single-step checks reaches the stopping boundary at lower expected cost than one using only full replays if and only if*

$$\frac{c_S}{c_F} < \frac{D_e^S}{D_e}, \quad (11)$$

where e is the true state of the cell.

Since e is unknown, the proposition gives a clear rule only when the cost ratio lies outside the interval spanned by the two critical ratios; inside it, which fidelity is better depends on the posterior, which is what CARVE’s look-ahead weighs.

With our default parameters ($p_1 = 0.54$, $b = 0.05$, $s = 0.85$, $\epsilon = 0.15$) the critical ratio is 0.31 for true edges and 0.44 for false ones; with a weak check ($s = 0.65$, $\epsilon = 0.35$) it falls to 0.05–0.08. Section D.4 tests both predictions.

D.4 Controlled Experiments

Real agents do not reveal which component truly caused a failure, so we first evaluate in a testbed where the causal graph is known and the method’s model is deliberately misspecified.

Setup

Testbed. Each world has $I = 12$ categories with Zipfian frequencies, $J = 20$ components and $K = 3$ patches per cell. Every category has one or two true components and one *decoy* component that wrong attributions prefer, mimicking symptom-level mislabelling. Patch quality is bimodal: with probability 0.3 a patch is bad ($q \sim \text{Beta}(1, 9)$), otherwise $q \sim \text{Beta}(6, 3)$. A trajectory sampled for category i is mislabelled with probability $1 - \lambda = 0.2$, in which case it comes from another category (weighted by frequency) and is repaired only if its own edge to j exists, unlike the model’s flat treatment of mislabelled traces. The analyst attributes 20 failures per category with accuracy $r = 0.5$; wrong attributions go to the decoy with probability 0.6. The analyst’s accuracy is estimated from an audit of 20 attributions. Costs are $c_F = 1$, $c_S = 0.1$, $c_{fp} = 0.02$, and $s = 0.85$, $\epsilon = 0.15$, $b = 0.05$. Section D.4 relaxes the assumptions the generator shares with the model. Full details are in Appendix D.6.

Method	Budget (full-replay units)			
	20	50	100	200
LLM attribution only	0.297 $\pm$ 0.021	0.297 $\pm$ 0.021	0.297 $\pm$ 0.021	0.297 $\pm$ 0.021
Replay-each (do-then-verify)	0.255 $\pm$ 0.021	0.270 $\pm$ 0.021	0.466 $\pm$ 0.027	0.468 $\pm$ 0.026
Replay-each, 3 patches	0.301 $\pm$ 0.025	0.317 $\pm$ 0.026	0.401 $\pm$ 0.026	0.612 $\pm$ 0.026
Sequential-each (adaptive stopping)	0.426 $\pm$ 0.016	0.522 $\pm$ 0.018	0.643 $\pm$ 0.016	0.813 $\pm$ 0.014
Thompson sampling	0.407 $\pm$ 0.020	0.419 $\pm$ 0.020	0.500 $\pm$ 0.017	0.625 $\pm$ 0.014
Uncertainty sampling	0.455 $\pm$ 0.016	0.581 $\pm$ 0.019	0.690 $\pm$ 0.017	0.804 $\pm$ 0.016
Uncertainty + cheap checks	0.549 $\pm$ 0.018	0.662 $\pm$ 0.016	0.784 $\pm$ 0.015	0.876 $\pm$ 0.009
CARVE (all components)	0.666 $\pm$ 0.019	0.787 $\pm$ 0.015	0.853 $\pm$ 0.011	0.902 $\pm$ 0.009
full replay only	0.514 $\pm$ 0.016	0.609 $\pm$ 0.020	0.697 $\pm$ 0.017	0.825 $\pm$ 0.012
myopic ($m=1$ look-ahead)	0.591 $\pm$ 0.016	0.652 $\pm$ 0.014	0.706 $\pm$ 0.016	0.729 $\pm$ 0.014
uninformed prior	0.347 $\pm$ 0.019	0.474 $\pm$ 0.017	0.631 $\pm$ 0.015	0.800 $\pm$ 0.012
one patch per cell	0.571 $\pm$ 0.019	0.636 $\pm$ 0.020	0.673 $\pm$ 0.019	0.688 $\pm$ 0.020
no label-noise term	0.663 $\pm$ 0.020	0.772 $\pm$ 0.018	0.864 $\pm$ 0.012	0.922 $\pm$ 0.006

Table 5: Net gain as a fraction of the oracle (mean $\pm$ s.e., 50 worlds). The lower block ablates CARVE.

Baselines. *LLM-only* accepts, for every category, the most-attributed component and any component holding at least 40% of the attributions, with the first patch, i.e. it acts on an induced taxonomy without verification. *Replay-each* tests attributed hypotheses in order of attribution count with three full replays of the first patch and accepts a hypothesis if at least two succeed, mirroring do-then-verify methods such as DoVer; *Replay-each, 3 patches* gives each hypothesis two replays of each of three patches and accepts if any patch succeeds twice; *Sequential-each* replays each hypothesis, cycling through patches, until CARVE’s own posterior leaves $[0.05, 0.95]$. Untested hypotheses fall back to LLM-only, except for *Sequential-each*, which uses the Bayes decision. *Thompson sampling* samples (e, q) from the posterior and replays the cell and patch with the largest sampled repair. *Uncertainty sampling* replays the cell with the largest frequency-weighted uncertainty $f_i \min(P_{ij}, 1 - P_{ij})$, cycling through patches, at full fidelity; *Uncertainty + cheap checks* first runs three single-step checks per (cell, patch) and then confirms with full replays. All baselines except LLM-only and the *Replay-each* variants share CARVE’s posterior and decision rule, so comparisons among them isolate the selection rule.

Metrics. Net gain (Eq. 3) as a fraction of the oracle, and edge F1. We report means and standard errors over 50 worlds for the main experiment and 30 for the others; all methods see the same worlds, so differences are paired.

Main results Figure 2 and Table 5 show that CARVE is best at every budget. With 50 units it recovers 79% of the oracle gain; the strongest baseline, uncertainty sampling with the same cheap checks, recovers 66% and needs about twice the budget (100 units, 78%) to match it. The gap nar-

rows at large budgets (90% vs. 88% at 200), as every reasonable method eventually resolves the frequent categories.

Where the gain comes from. Two comparisons separate the selection rule from the cheap checks. With full replays only, CARVE beats uncertainty sampling by 1–6 points (e.g. 0.609 vs. 0.581 at budget 50): the knowledge-gradient rule alone helps only modestly. Given cheap checks, its advantage grows to 7–13 points (0.787 vs. 0.662 at 50), because it decides per action whether a check is informative enough for its price and stops checking a cell once checks can no longer change the decision, whereas a fixed check-then-confirm schedule wastes checks on resolved cells. Most of CARVE’s advantage is therefore in *how it uses* cheap checks, which also makes their calibration critical (§D.4).

Verification baselines. LLM-only is stuck at 30% because it accepts decoys and bad first patches. *Replay-each* is initially *worse* than not verifying: with $p_1 \approx 0.54$, requiring two successes in three replays rejects about 44% of true edges even when the patch is good, and its fixed per-hypothesis cost exhausts small budgets on frequent but already-certain hypotheses. Testing three patches (61% at 200) or stopping adaptively (81% at 200) helps, but both remain well below CARVE at small budgets (32% and 52% at 50). Thompson sampling spends most replays on cells it already believes in and reaches only 63% at 200.

Ablations. Removing single-step checks costs 15–18 points at small budgets. Restricting the look-ahead to one observation is below the full method at every budget and plateaus at 73%, because many cells cannot be flipped by a single outcome and are never revisited. A flat prior costs 32 points at budget 20 and still 10 points at 200. With one patch per cell the method plateaus at 69%: it cannot tell a wrong component from a bad patch and rejects true edges for good. Dropping the label-noise term makes no difference at the default label accuracy $\lambda = 0.8$ (it is even slightly better at large budgets, 0.922 vs. 0.902 at 200), but it matters when labels are worse (§D.4).

When does the LLM prior help? Figure 3 varies the analyst’s accuracy r . As Proposition 3 suggests, CARVE improves monotonically with r and is never below the flat-prior variant: at $r = 0.1$ it scores 0.51 ± 0.02 against 0.48 ± 0.03 . The prior is not exactly flat there (the decoy still receives

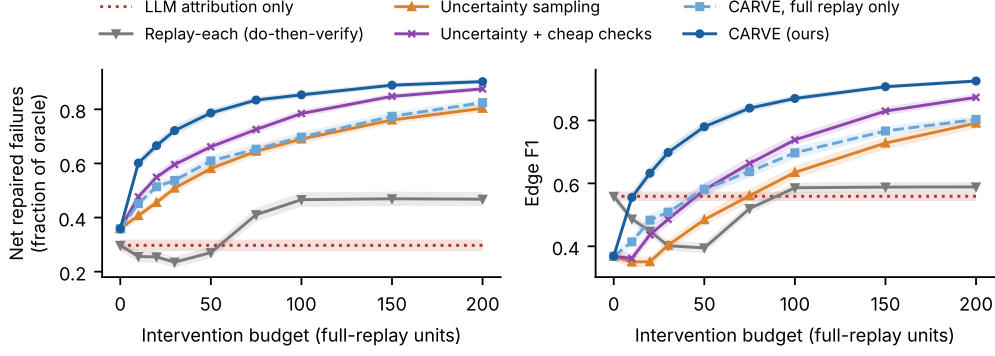


Figure 2: Net repaired failure mass (left) and edge F1 (right) against intervention budget, 50 worlds, shaded ± 1 s.e.

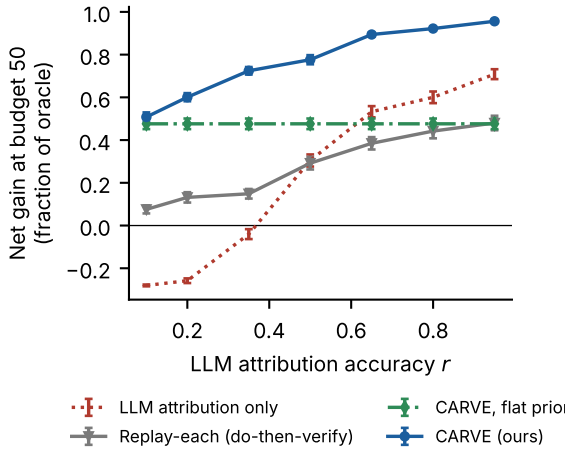


Figure 3: Net gain at budget 50 as the analyst’s attribution accuracy r varies (30 worlds, ± 1 s.e.).

about twice the base rate), but its shifts are small, so evidence quickly dominates. Acting on attributions directly is harmful below $r \approx 0.35$, where LLM-only has a *negative* net gain (regressions outweigh repairs), and even at $r = 0.95$ it reaches only 71% of the oracle against 96% for CARVE, since it cannot detect bad patches. Replay-each improves with r and only matches the flat-prior variant of CARVE at $r = 0.95$ (0.48 vs. 0.48).

When do cheap checks help? Figure 5 (Appendix D.8) is qualitatively consistent with Proposition 4. With the default check, the benefit of cheap checks shrinks as their cost grows and vanishes between 0.35 and 0.5 (0.64 ± 0.02 vs. 0.61 ± 0.03 at 0.35, overlapping; 0.59 vs. 0.61 at 0.5), in line with the predicted critical ratios of 0.31 and 0.44. At cost 0.1, a check of quality $s - \epsilon = 0.5$ (critical ratio 0.14–0.22) still helps, while one of quality 0.3 (critical ratio 0.05–0.08) brings no gain. At cost

Condition (budget 50)	Unc. + checks	CARVE
Default testbed	0.656 ± 0.022	0.776 ± 0.021
Label accuracy $\lambda = 0.5$	0.513 ± 0.023	0.688 ± 0.023
Analyst overconfident ($\hat{r} + 0.3$)	0.752 ± 0.018	0.857 ± 0.014
Analyst underconfident ($\hat{r} - 0.3$)	0.495 ± 0.023	0.653 ± 0.032
Judge fooled by bad patches	0.579 ± 0.022	0.377 ± 0.027
Regression cost $c_{fp} = 0.005$	0.711 ± 0.022	0.798 ± 0.016
Regression cost $c_{fp} = 0.05$	0.643 ± 0.024	0.793 ± 0.019
Regression cost $c_{fp} = 0.1$	0.624 ± 0.023	0.823 ± 0.018

Table 6: Violations of the model’s assumptions (30 worlds). The judge-fooled row is the one setting where CARVE loses.

0.5 CARVE is slightly below its full-replay-only variant although its action set contains every full-replay action, which shows that the greedy rule is not exactly cost-optimal when the two fidelities are nearly equally efficient.

Robustness The generator shares several assumptions with the model: check errors depend only on the full outcome, cells are independent, the analyst’s accuracy is estimated without bias, and the regression cost is small. Table 6 and Appendix D.8 relax each in turn.

Worse labels. With label accuracy $\lambda = 0.5$ both methods lose ground, and the label-noise term now matters: CARVE scores 0.688, against 0.588 without the term and 0.513 for the baseline.

Biased analyst audit. If the audit overstates the analyst’s accuracy by 0.3, CARVE actually improves (0.857), because a stronger prior concentrates the budget and the evidence corrects the decoys. Understating it by 0.3 costs 12 points (0.653), still above the baseline. Under-trusting a decent analyst is thus the more costly error in this testbed.

Regression cost. Raising c_{fp} from 0.005 to 0.1 leaves CARVE between 0.78 and 0.82, while LLM-

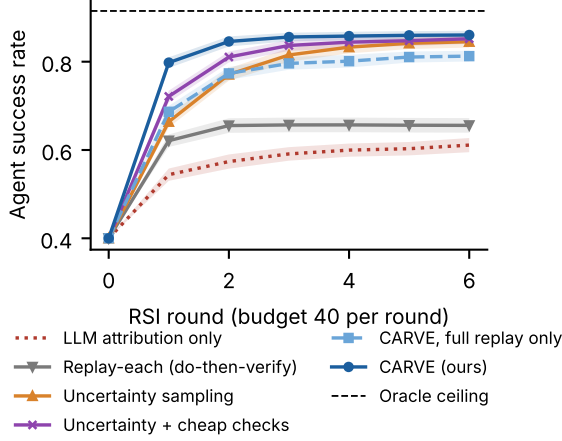


Figure 4: Agent success over six rounds of attribute-verify-patch, with a budget of 40 units per round (30 worlds, ± 1 s.e.).

only falls from 0.39 to -0.13 : the more costly a wrong change, the more verification is worth.

Joint causes. When 30% of categories need two components changed together (a single patch then repairs only a fifth as often), violating cell independence, CARVE drops from 0.787 to 0.661 at budget 50 but stays ahead of uncertainty sampling with checks (0.558; Table 9).

Correlated check errors. The one setting where CARVE loses is when the single-step judge is fooled by plausible-looking bad patches (false-positive rate 0.6 for patches with $q < 0.3$). CARVE then trusts checks that systematically pass bad patches and falls to 0.377, below every method that relies on full replays (0.58–0.61). Modelling the dependence (a quality-dependent false-positive rate in the likelihood) restores it to 0.584, on par with full replays, at a cost of 9 points when the judge is not fooled (0.684 vs. 0.776). Whether real judges behave this way is therefore the most important quantity to measure before relying on cheap checks.

Nuisance parameters and scale. Misspecifying b , λ or the patch-quality prior changes the result by at most 5 points (Table 10, Appendix D.8); misstating the check’s sensitivity and false-positive rate costs 6–14 points. On a larger graph ($I = 20$, $J = 50$; Table 8) the gap widens: at budget 50 CARVE reaches 78% of the oracle against 60% for uncertainty sampling with checks.

Inside a self-improvement loop Finally we embed each method in a loop. The agent starts at 40% success. In each round the analyst attributes 240 new failures, sampled in proportion to the current failure distribution; the method spends 40 units; accepted patches are applied, which repairs failures (true edges) or adds a regression of c_{fp} times the current failure mass (false edges); and the failure distribution is recomputed, so priorities shift towards categories that remain. Posteriors carry over between rounds.

Figure 4 shows that CARVE reaches 80% success after one round and 85% after two, close to its plateau of 86% (the oracle ceiling is 91.5%). Uncertainty sampling with checks needs four rounds to reach 84% and ends at 85%; without checks it needs six. LLM-only and Replay-each plateau at 61% and 66%: once their accepted false edges and rejected true edges are fixed in place, new attributions do not correct them.

D.5 Proofs for CARVE’s propositions

Proposition 3. Let $L_t = \ell + \sum_{u \leq t} \log \frac{P(o_u|e=1)}{P(o_u|e=0)}$ be the posterior log-odds after t observations. Under $e = 1$, the increments are i.i.d. with mean $D_1 = \text{KL}(p_1||p_0) > 0$. The rule stops at $N = \inf\{t : L_t \notin (-A, A)\}$. By Wald’s identity, $\mathbb{E}_1[L_N - \ell] = D_1 \mathbb{E}_1[N]$. Wald’s approximations give $P_1(L_N \leq -A) \approx e^{-(A+\ell)} \approx \delta e^{-\ell}$ and $P_0(L_N \geq A) \approx \delta e^{\ell}$. Neglecting overshoot and this probability (which requires $\delta e^{|\ell|} \ll 1$), $L_N = A$, hence $\mathbb{E}_1[N] = (A - \ell)/D_1$; if $\ell \geq A$ no observation is taken, which is why we assume $|\ell| < A$. Symmetrically, under $e = 0$ the drift is $-D_0$ and $\mathbb{E}_0[N] = (A + \ell)/D_0$. Taking the difference between ℓ_ρ and ℓ_{ij} and averaging over cells with $P(e=1) = \rho$ gives Δ . When the patch quality q is unknown, outcomes are exchangeable rather than i.i.d. under the marginal likelihood, and Wald’s identity does not apply directly; conditioning on q recovers the result with $D_1 = D_1(q)$, which approaches zero for a bad patch. This is the formal reason a single bad patch can stall verification and why CARVE keeps several patches per cell.

Proposition 4. By the same argument a single-step test needs $(A \mp \ell)/D_e^S$ observations at cost c_S each, and a full-replay test needs $(A \mp \ell)/D_e$ at cost c_F . The single-step test is cheaper iff $c_S/D_e^S < c_F/D_e$. Since the single-step outcome is a garbling of the full outcome, the data-processing inequality

gives $D_e^S \leq D_e$, so the critical ratio is at most one.

D.6 Simulation Details

Category frequencies are $f_i \propto 1/i$. For each category the number of true components is uniform on $\{1, 2\}$ and the decoy is a further distinct component. The audit estimate $\hat{r}$ is a binomial proportion from 20 audited attributions. The prior uses $d = 1.5$. The model’s grid has 101 points; look-ahead horizons are $\{1, 2, 4, 8\}$; the stopping threshold is 10^{-6} . Replay-each uses three replays and accepts with at least two successes. Uncertainty sampling switches to an untried patch once the current patch has three full observations. In the loop experiment, 240 attributions per round are allocated to categories by a multinomial draw with the current failure distribution. All randomness is seeded; the code reproduces every number in this paper with `python -m sim.experiments all`.

D.7 Checking Proposition 1

Table 7 compares the predicted expected number of observations with simulated sequential tests ($p_1 = 0.543$, $p_0 = 0.05$, $\delta = 0.02$, 4000 runs per row). The predictions capture the direction and approximate size of the effect of the initial log-odds; they are least accurate when $|\ell|$ is large, where overshoot and wrong-boundary exits (last column) are no longer negligible.

ℓ	e	predicted N	simulated N	wrong exit
-3	1	7.17	5.94	0.242
-3	0	1.55	2.54	0.000
-2	1	6.13	6.28	0.117
-2	0	3.28	3.73	0.001
-1	1	5.09	5.79	0.047
-1	0	5.02	5.21	0.003
+0	1	4.05	4.76	0.012
+0	0	6.75	7.28	0.009
+1	1	3.01	4.27	0.006
+1	0	8.48	8.58	0.015
+2	1	1.97	2.97	0.001
+2	0	10.22	10.23	0.068

Table 7: Proposition 3: predicted vs. simulated observations to a decision.

D.8 Additional Results

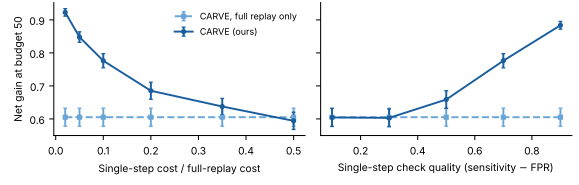


Figure 5: Effect of the single-step check’s cost (left, quality fixed at $s=0.85$, $\epsilon=0.15$) and quality (right, cost fixed at 0.1) on net gain at budget 50.

Method	50	100	300
LLM attribution only	0.231 $\pm$ 0.033	0.231 $\pm$ 0.033	0.231 $\pm$ 0.033
Replay-each (do-then-verify)	0.224 $\pm$ 0.034	0.400 $\pm$ 0.037	0.531 $\pm$ 0.032
Uncertainty sampling	0.502 $\pm$ 0.022	0.637 $\pm$ 0.020	0.822 $\pm$ 0.015
Uncertainty + cheap checks	0.603 $\pm$ 0.019	0.746 $\pm$ 0.018	0.873 $\pm$ 0.013
CARVE, full replay only	0.613 $\pm$ 0.013	0.710 $\pm$ 0.012	0.834 $\pm$ 0.010
CARVE (ours)	0.780 $\pm$ 0.016	0.860 $\pm$ 0.011	0.933 $\pm$ 0.008

Table 8: Net gain on a larger graph ($I=20$, $J=50$), 20 worlds.

Method	20	50	100	200
LLM attribution only	0.228 $\pm$ 0.028	0.228 $\pm$ 0.028	0.228 $\pm$ 0.028	0.228 $\pm$ 0.028
Replay-each	0.202 $\pm$ 0.027	0.207 $\pm$ 0.023	0.348 $\pm$ 0.033	0.362 $\pm$ 0.036
Thompson sampling	0.325 $\pm$ 0.031	0.343 $\pm$ 0.032	0.398 $\pm$ 0.032	0.484 $\pm$ 0.028
Uncertainty sampling	0.345 $\pm$ 0.036	0.465 $\pm$ 0.035	0.544 $\pm$ 0.035	0.662 $\pm$ 0.029
Uncertainty + cheap checks	0.423 $\pm$ 0.037	0.558 $\pm$ 0.035	0.615 $\pm$ 0.034	0.685 $\pm$ 0.030
CARVE (ours)	0.573 $\pm$ 0.029	0.661 $\pm$ 0.029	0.701 $\pm$ 0.027	0.745 $\pm$ 0.026

Table 9: Net gain when 30% of categories need two components changed together (30 worlds).

Model assumption (true value)	Net gain
All nuisance parameters as in the generator	0.776 $\pm$ 0.021
Spurious rate b : 0.01 (0.05)	0.811 $\pm$ 0.012
Spurious rate b : 0.15 (0.05)	0.826 $\pm$ 0.016
Label accuracy λ : 1.0 (0.8)	0.768 $\pm$ 0.022
Label accuracy λ : 0.6 (0.8)	0.794 $\pm$ 0.017
Check s/ϵ : .95/.05 (.85/.15)	0.717 $\pm$ 0.016
Check s/ϵ : .70/.30 (.85/.15)	0.636 $\pm$ 0.021
Patch prior Beta(1, 1) (Beta(2, 2))	0.811 $\pm$ 0.015
Patch prior Beta(5, 2) (Beta(2, 2))	0.777 $\pm$ 0.020

Table 10: CARVE with misspecified nuisance parameters, budget 50, 30 worlds. Several “wrong” values do slightly better than the generator’s, a reminder that the model is misspecified in other ways too.